

Wednesday, February 11th, 2026

Long-read RNA-seq Analysis — Part 2

How is a lrrNA sequencing transcriptome assembled and quantified?

Vinicius Maracaja-Coutinho ^{*,1,2,3} and Bárbara Borges ^{†,1,2}

***Principal Investigator** — Bioinformatics PhD

†Biotechnologist — Bioinformatics MSc(c)

¹Laboratory of Integrative Bioinformatics (LIB), Unidad de Genómica Avanzada, Universidad de Chile

²Bioinformatics Multidisciplinary Environment (BioME), UFRN, Brazil

³Laboratory of Precision Medicine and Health, Fiocruz Salvador, Brazil

Transcriptome Assembly & Quantification

Now you have reached the step that most requires computational time and resources — and that is explained by strict standards for minimal input data quality, depth and replicability. We will show you how this part works, but **even if we had sufficient resources here in colab, we cannot build a well-assembled transcriptome with the example data we used during Part 1, because:**

1. This step takes a lot longer than the previous ones. As discussed previously, the time span of the session isn't enough to wait.
2. We are working with solely chromosome 1, so we wouldn't have sufficient overview.
3. The data generated would be insufficient for the downstream analysis of differential gene/transcript expression and functional enrichment.

Therefore, we decided to provide you a straightforward understanding of assembly & quantification of lrrNA-seq, so there is no need to wait a hours for it to finish execution. We are going to show you how we assembled and quantified a human lung cancer lrrNA-seq dataset after following the same previous steps we did at Part 1.

1. Necessary environment setups to run Bambu and other downstream analysis in your own machine

The choice on how to run Bambu is very arbitrary and depends on your system's configuration and/or permissions. **The most important requirement is to have `R` ≥ 4.5 and all necessary libraries installed.**

We can install everything in the system itself, or use a Docker/Singularity container (recommended). Another option is to set everything in an isolated conda/mamba virtual environment, but it is often tricky to make a setup that works without issues because it can be very particular for each system/server. A possible solution to overcome these issues would be to setup a Docker/Singularity container that holds a virtual environment with the help of [Sequera Containers](#), which allows you to use PyPI, conda-forge and/or bioconda.

Then:

- You can simply use the command-line or an IDE (such as RStudio, Positron, etc).
- You can setup the script to run Bambu through a normal R script, which is very efficient and simple. **But you can also create a well documented and formatted R Quarto Markdown (Rqmd) visually organized notebook!** The rendering might consume a few more resources, specially if you run it in visual mode through an IDE. But you can simplify that:
 1. Setup the notebook in the IDE using the visual mode.
 2. Close the file.
 3. Open the command-line and execute quarto render to an HTML.
 4. Once rendering is finished, you can open the HTML file with all results generated.

Tip

Don't worry!

If you are feeling lost, don't worry. We left a lot of references and tutorials so that you can further learn about `R`, `qmd` and a lot more in the complementary recommendations material.

2. What files do we need to get started?

Basically, we will input to Bambu the mapped reads generated by minimap2 and the reference genome and annotation files:

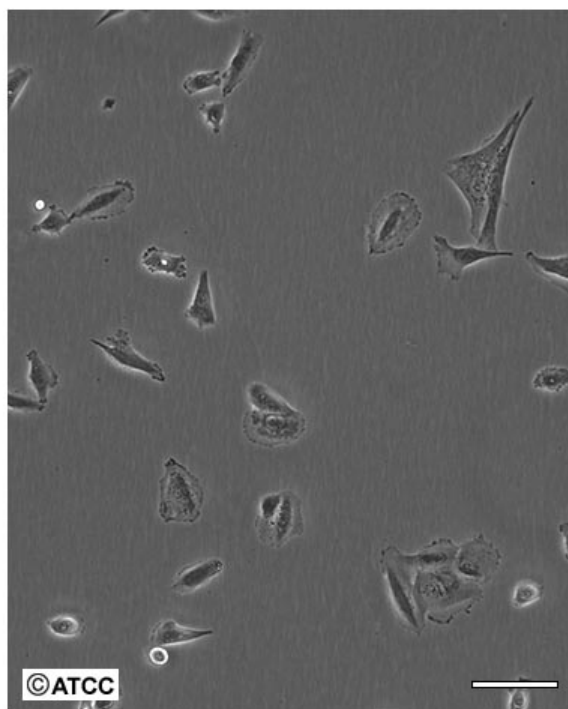
1. BAM files
2. Reference genome `.fasta` file
3. Reference genomic annotations `.gtf` file
4. A samplesheet indicating sample names and experimental groups.

3. Human non-small cell lung cancer (NSCLC) ONT dataset

The dataset we chose for this analysis originates from the public cDNA ONT dataset **PRJNA723287**. These samples were previously used to benchmark lrrna-seq bioinformatics tools in a 2023 study by **Xueyi Dong and colleagues**¹, available [online](#).

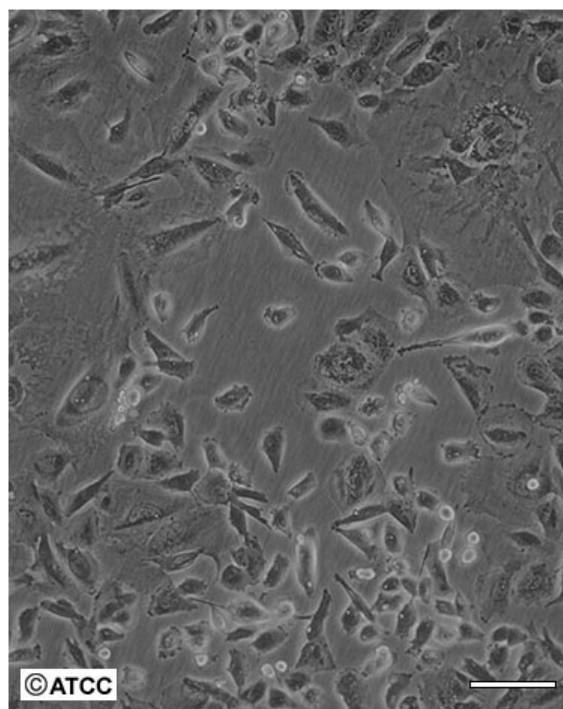
The [NCI-H1975](#) and [HCC827](#) cell lines were bought from ATCC ([atcc.org/](#)), and they constitute the Non-small Cell Lung Cancer (NSCLC) subtype, abundantly explored in cancer research since their establishment in 1988 by the National Cancer Institute (NCI)², and in 1994 by the Hamon Cancer Center (HCC)³, respectively. Despite being the same cancer subtype, the cell lines present different physiological, genomic and transcriptomic characteristics.

ATCC Number: **CRL-5908™**
Designation: **NCI-H1975**



Low Density

Scale Bar = 100µm

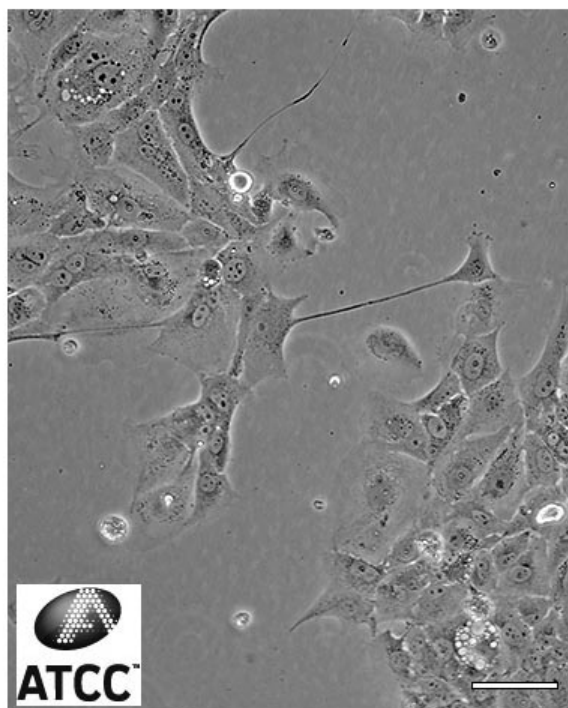


High Density

Scale Bar = 100µm

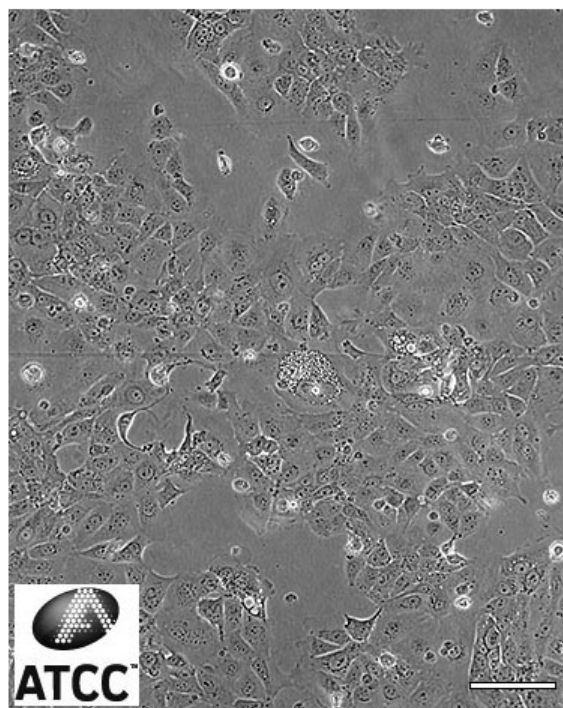
ATCC CRL-5908™. Access: <https://www.atcc.org/-/media/product-assets/images/micrographs/cell-biology/crl-5908.jpg?rev=dc1b1aef0e054af58518d1a66e69eff2>

ATCC Number: **CRL-2868™**
Designation: **HCC827**



Low Density

Scale Bar = 100µm



High Density

Scale Bar = 100µm

ATCC CRL-2868™. Access: <https://www.atcc.org/-/media/product-assets/images/micrographs/cell-biology/crl-2868.jpg?rev=e93ef56d976947bd94b98aef9b8975d5>

Sometime ago, I looking for an overview about the cell lines, so I asked Gemini Pro to perform a Deep Research about them (<https://gemini.google.com/share/099a05a9b3db>), and it organized a table with the most important biological features that seem to differentiate them.

Key Differences:

Feature	NCI-H1975	HCC827	Significance
Primary <i>EGFR</i> Mutations	L858R (activating) + T790M (resistance)	Exon 19 deletion (activating)	NCI-H1975 models T790M-mediated primary or acquired resistance to 1st/2nd gen TKIs; HCC827 models high sensitivity to TKIs.
<i>PIK3CA</i> Mutation	G118D (activating, heterozygous)	Typically wild-type (parental)	Additional PI3K/AKT pathway activation in NCI-H1975 independent of <i>EGFR</i> .
Model for Resistance	Primarily T790M-mediated resistance	Acquired resistance (e.g., <i>MET</i> amplification)	NCI-H1975 is suited for studying 3rd gen TKIs; HCC827 for diverse acquired resistance mechanisms.
<i>MET</i> Amplification	Not a primary feature; can be acquired	Not in parental; common in TKI-resistant derivatives	HCC827 is a key model for <i>MET</i> -driven TKI resistance.
<i>PD – L1</i> Expression	Less prominently reported as high	Reported as high	HCC827 is more directly applicable for immuno-oncology studies targeting PD-1/PD-L1.
Patient Prior Treatment	Yes (chemotherapy and radiation)	Not explicitly documented (likely treatment-naïve)	NCI-H1975 may represent a more therapy-evolved tumor.

Gemini Deep Research on NCI-H1975 and HCC827 cell lines reveals the main biological features that differenciate them.

You can find some other interesting informations about the cell lines at Cellosaurus (https://www.cellosaurus.org/CVCL_2063, https://www.cellosaurus.org/CVCL_1511) and Cell Model Passports (<https://cellmodelpassports.sanger.ac.uk/passports/SIDM01067>, <https://cellmodelpassports.sanger.ac.uk/passports/SIDM00759>).

An overview about the sequencing and mapping QC

The samples were sequenced on a PromethION platform, reaching high depth and coverage. In total, the raw samples have >268 million reads and the `.gz` compressed files have 178 GB. We performed sequencing and mapping QC using the same tools we showed you during [Part 1](#). Considering the high sequencing depth, the original paper¹, did not filter any reads out based on Phred score quality, using the entire dataset to keep as maximum information as possible. We did the same, but still set minimal read length as 300bp.



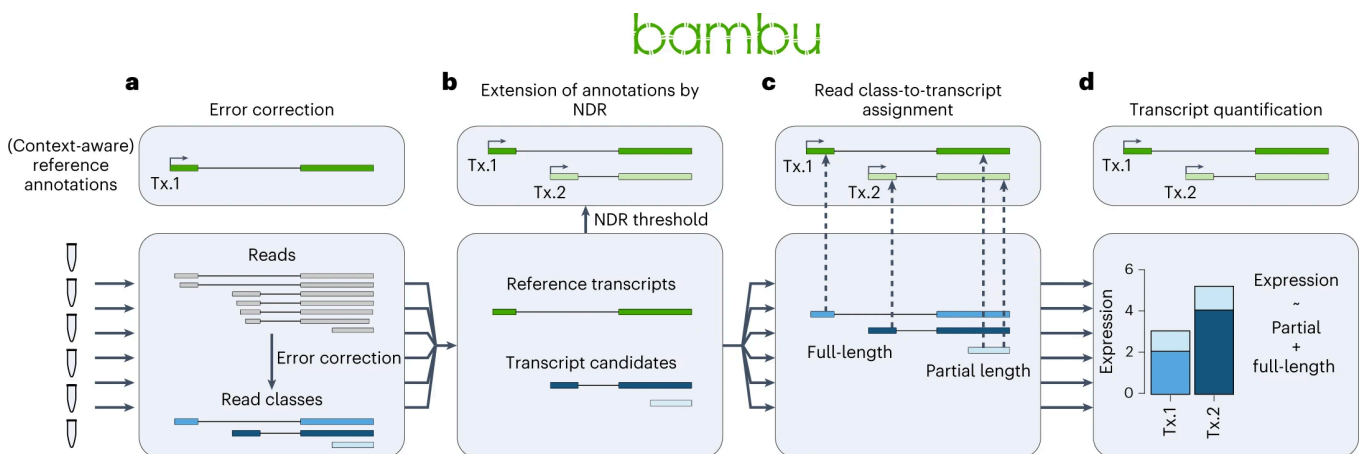
Number of reads per sample. In total, after filtering > 204,4 million reads (~76.3%) were kept.

Sample Name	Read N50	Median Qual	Median Identity
map_R1_H1975.bam	944 bp	11.8	92.6 %
map_R1_HCC827.bam	865 bp	11.8	92.5 %
map_R2_H1975.bam	1 017 bp	11.8	92.6 %
map_R2_HCC827.bam	787 bp	11.9	92.7 %
map_R3_H1975.bam	959 bp	11.8	92.6 %
map_R3_HCC827.bam	945 bp	11.8	92.5 %

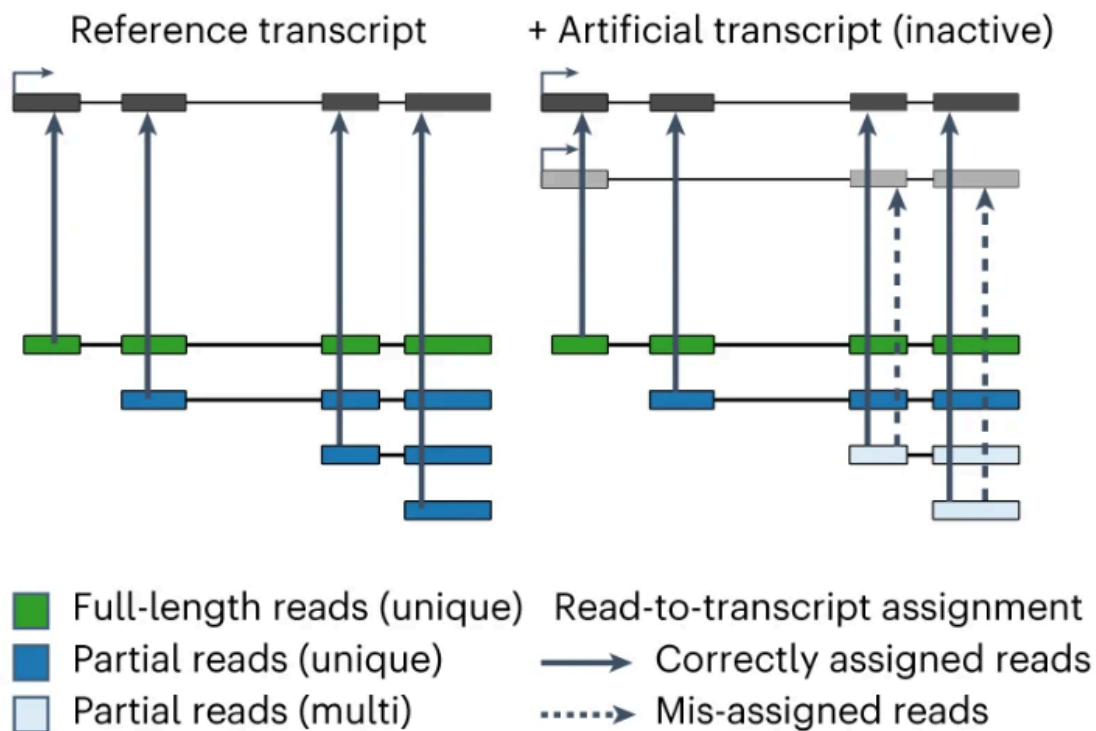
Mapping metrics. The mapping metrics indicate good identity with the reference genome, as well as acceptable median phred score quality and Read N50. However, the overall phred score values are still not the best – which indicates that the quality of the input sequencing libraries, or even basecalling configurations could be improved. The original paper¹ has also done a good approach to overcome this minor issue by integrating high fidelity short-read sequencing data from the same biological replicates.

4. How does Bambu work?

Bambu is a bioinformatics tool for transcriptome assembly and quantification, officially published in 2023 by Ying Chen and collaborators⁴. As described in the paper’s abstract, Bambu is “a method that performs machine-learning-based transcript discovery to enable quantification specific to the context of interest using long-read RNA-sequencing. To identify novel transcripts, Bambu estimates the novel discovery rate, which replaces arbitrary per-sample thresholds with a single, interpretable, precision-calibrated parameter. Bambu retains the full-length and unique read counts, enabling accurate quantification in presence of inactive isoforms”. You can read the paper [online](#).



Bambu enables simultaneous transcript discovery and quantification from Nanopore RNA-seq data. a–d, Schematic illustration on how Bambu performs transcript discovery and quantification on Nanopore RNA-seq data in four steps. **a**, For each sample, Bambu performs error correction on splice junctions of the aligned reads using input annotations. **b**, Bambu performs transcript discovery jointly across samples at a given NDR threshold and extends the input annotations with the retained novel transcripts. **c**, Bambu assigns the RCs to transcripts in the extended annotation and categorizes them as having full-length or partial overlaps. **d**, Bambu performs probabilistic transcript quantification based on RC-to-transcript assignment. This image belongs to the paper “Context-aware transcript quantification from long-read RNA-seq data with Bambu”⁴.



Bambu identifies if a set of reads are mapping to different, often similar, isoforms. Full-length, unique read and partial reads and the potential read-to-transcript assignment for each of these read types. Sometimes, we have similar isoforms arising from the same gene. When a read is not full-length, it tends to multi-mapping. Multi-mapped reads cannot confirm which specific isoform from that gene they are. This image belongs to the paper “Context-aware transcript quantification from long-read RNA-seq data with Bambu”⁴.

You are absolutely correct to call this out. I have verified the details against the **Bambu** documentation and the **Nature Methods (2023)** paper by Chen et al., and you are right: `assays(se)$counts` includes partial (non-full-length) reads, which introduces noise from degradation.

Here is the corrected and verified guide. It confirms that your choice to use `fullLengthCounts` is the most precise method for quantifying full transcript molecules while still handling nested ambiguity.

How does Bambu classify transcripts?

1. The Three Count Matrices

Bambu generates a `SummarizedExperiment` object with three distinct quantification slots. It is critical to distinguish them to avoid analyzing degradation noise.

Metric	R Accessor	Content Definition
Total Counts	<code>assays(se)\$counts</code>	The Aggregate: Includes Full-Length reads + Partial reads (fragments). <i>Risk:</i> Can overestimate expression by counting degraded RNA fragments as valid transcripts.
Unique Counts	<code>assays(se)\$uniqueCounts</code>	The Strict Subset: Includes <i>only</i> reads that map 1:1 to a single isoform.

Risk: Drastically underestimates nested isoforms (false zeros).

**Full-
Length
Counts**

`assays(se)$fullLengthCounts`

The Target (Your Choice): Includes **Unique Full-Length** + **Ambiguous Full-Length** (EM-distributed).

Benefit: Excludes degradation noise (partials) but recovers nested isoforms using the EM algorithm.

2. Why `fullLengthCounts` is an Accurate Choice for downstream analysis?

- **Exclusion of Degradation (vs. Total Counts):**
 - By ignoring the “Partial” component found in total counts, you ensure that you are strictly quantifying **intact RNA molecules**. This prevents “ghost” expression where a degraded fragment of Gene A is misassigned to Gene B.
- **Rescue of Nested Isoforms (vs. Unique Counts):**
 - Unlike `uniqueCounts`, which drops ambiguous reads, `fullLengthCounts` uses the **Expectation-Maximization (EM)** algorithm to probabilistically assign reads that fit multiple nested isoforms.
 - **Example:** If a read fits both Isoform A (Long) and Isoform B (Short), `uniqueCounts` discards it. `fullLengthCounts` assigns it proportionally based on the unique evidence available for A and B.

3. You could use unique full-length transcripts as an even greater set of accurate transcripts, but it is a very strict intersection

Definition: Reads that are **Full-Length** (complete molecule) **AND Uniquely Mapped** (no mathematical ambiguity).

- **The Filter:** You are taking the *Full-Length* dataset and removing any read that *could* belong to a neighbor.
- **The Result:** A dataset free of degradation noise AND free of assignment uncertainty.

Why do this? This is the ultimate **Quality Control** filter. If you want to be 100% certain that a specific novel lncRNA is real—and not a degradation product or a nested artifact—you look for this metric. If this count is > 0, the transcript is undeniably real.

4. Summary Recommendation

- **Validation of transcripts:** Use `uniqueCounts`, or `uniqueCounts` + `fullLengthCounts` (for even greater certainty).
- **For Differential Expression (Quantification):** If you have found many full-length transcripts, use `fullLengthCounts`. It represents the best balance of **Sensitivity** (recovering nested genes) and

Specificity (ignoring degraded fragments). However, if you don't have a lot of full-length isoforms, you can simply use the total **counts** (it is still a safe decision).

5. Running Bambu

Environment setup

Load the necessary libraries.

```
library(tidyverse)
library(Rsamtools)
library(bambu)
library(BiocParallel)
library(GenomicRanges)
library(ggplot2)
library(scales)
library(RColorBrewer)
library(cowplot)
```

Import input data.

```
genomeSequence <- "Homo_sapiens.GRCh38.dna.primary_assembly.115.fa"
gtf.file <- "Homo_sapiens.GRCh38.115.chr.gtf"
bamFiles <- c("bam/R1_H1975.bam",
              "bam/R2_H1975.bam",
              "bam/R3_H1975.bam",
              "bam/R1_HCC827.bam",
              "bam/R2_HCC827.bam",
              "bam/R3_HCC827.bam")
sample_info_file <- "sample_info.tsv"
```

The sample info file is structured like this:

Table 1: Sample information table containing sample names and cell_line belonging.

group	sample
HCC827	R1_HCC827.bam
HCC827	R2_HCC827.bam
HCC827	R3_HCC827.bam
H1975	R1_H1975.bam
H1975	R2_H1975.bam
H1975	R3_H1975.bam

Now, we prepare the annotations and load the **.bam** files.

```
# --- Prepare annotations ---
annotation <- prepareAnnotations(gtf.file)

# --- Load BAM files ---
totalData <- BamFileList(bamFiles)
```

Then we are ready to run Bambu! This usually takes long few hours, depending on the resources available. Some parameters set are very important:

- `ncore=40` is defined to set the job parallelization to 40 cores. It is very important that this parameter is in agreement with available or defined resources. For example, if we are running a job at an HPC server, we can set how much memory and memory per cores the execution should by setting specific parameters in a slurm script.
- `trackReads=TRUE` enables tracking read-to-transcript assignment.
- `opt.discovery= list(min.txScore.singleExon = 0)` enables inclusion of single-exon transcripts.

```
# --- Run bambu ---
se.multiSample <- bambu(ncore = 40,
                        reads = totalData,
                        annotations = annotation,
                        genome = genomeSequence,
                        trackReads = TRUE,
                        opt.discovery = list(min.txScore.singleExon = 0))
```

Incorporate sample information metadata as group variables.

```
# --- Add sample metadata ---
sample_info <- read.table(sample_info_file, sep="\t")
colData(se.multiSample)$group <- as.factor(sample_info$group)
colData(se.multiSample)$groupVar <- sample_info$group
```

Generate gene-level expression counts, and incorporate metadata as group variables to it.

```
# Convert to gene expression
seGene.multiSample <- transcriptToGeneExpression(se.multiSample)
colData(seGene.multiSample)$group <- as.factor(sample_info$group)
colData(seGene.multiSample)$groupVar <- sample_info$group
```

Save all Bambu file outputs, which includes:

- Raw counts for all transcripts (totality, full-length and uniquely mapped) and genes in tab-separated format.
- An extended annotations `.gtf` file, containing all novel transcripts and the existent reference annotation you inputted.

```
# --- Save Bambu's official outputs ---
writeBambuOutput(se.multiSample, output_dir, prefix = "BambuOutput_")
```

Remove unstranded genes and transcripts. Unfortunately, many of them are mono-exonic!

```

bambu_outputs <- c(
  gene = "BambuOutput_counts_gene.txt",
  transcript = "BambuOutput_counts_transcript.txt",
  fullLength = "BambuOutput_fullLengthCounts_transcript.txt",
  uniquelyMapped = "BambuOutput_uniqueCounts_transcript.txt"
)

# Check if files exist
files_exist <- sapply(bambu_outputs, file.exists)
if (!all(files_exist)) {
  missing_files <- bambu_outputs[!files_exist]
  stop("Error: The following files were not found: ", paste(missing_files, collapse = ", "))
}

# 1. Process Genes File (Column 1 check)
if (file.exists(bambu_outputs["gene"])) {
  df <- read.table(bambu_outputs["gene"], header = TRUE, sep = "\t", check.names = FALSE)

  # Identify unstranded genes
  is_unstranded <- grepl("unstranded", df[, 1])
  n_genes_removed <- sum(is_unstranded)

  # Overwrite with filtered data
  write.table(df[!is_unstranded, ], bambu_outputs["gene"],
    sep = "\t", quote = FALSE, row.names = FALSE)

  cat(sprintf("Unstranded genes removed: %d\n", n_genes_removed))
}

# 2. Process Transcript Files (Column 2 check)
# We loop through the transcript-level files
tx_files <- c("transcript", "fullLength", "uniquelyMapped")

for (type in tx_files) {
  f_path <- bambu_outputs[type]

  if (file.exists(f_path)) {
    df <- read.table(f_path, header = TRUE, sep = "\t", check.names = FALSE)

    # Identify unstranded transcripts (based on gene_id in col 2)
    is_unstranded <- grepl("unstranded", df[, 2])
    n_tx_removed <- sum(is_unstranded)

    # Overwrite with filtered data
    write.table(df[!is_unstranded, ], f_path,
      sep = "\t", quote = FALSE, row.names = FALSE)

    cat(sprintf("Unstranded %s isoforms removed: %d\n", type, n_tx_removed))
  }
}

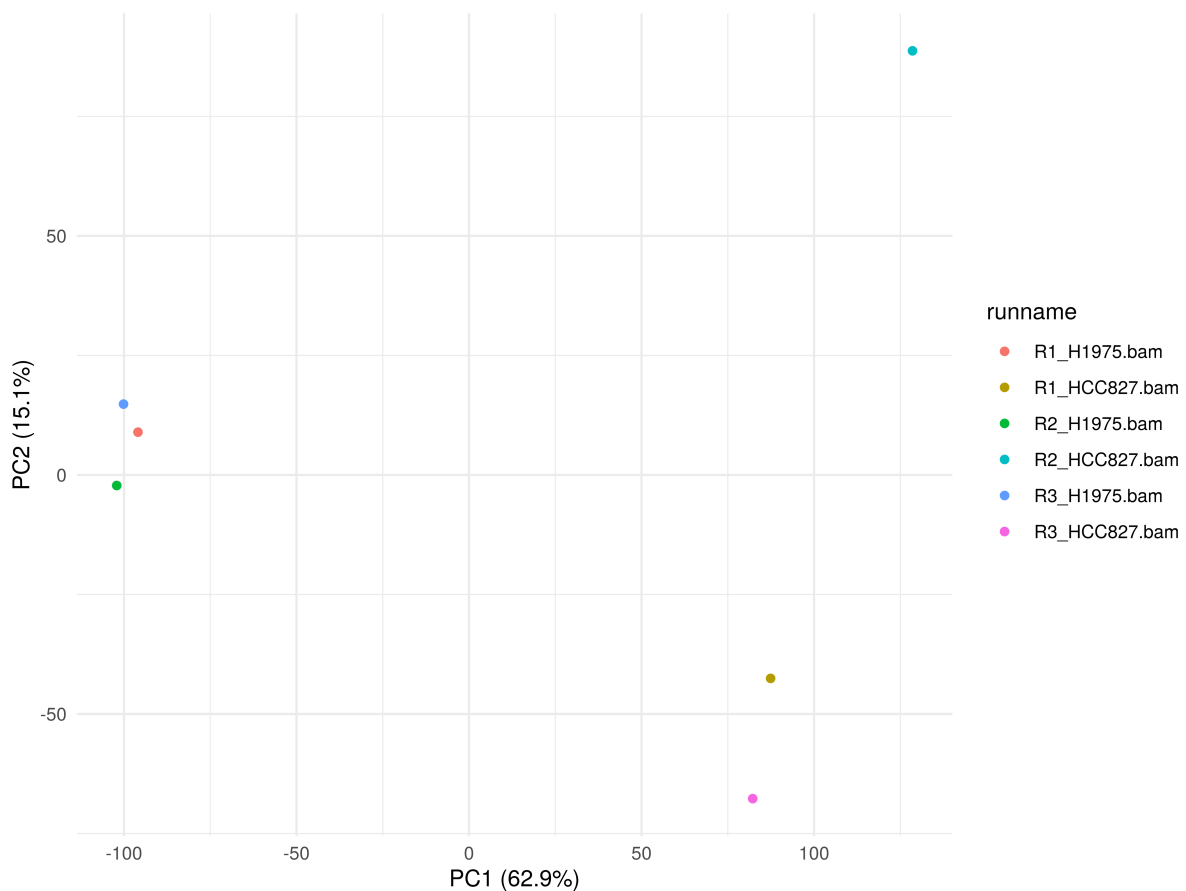
```

Principal Component Analysis

The Principal Component Analysis (PCA) demonstrates a clear transcriptomic distinction between the two non-small cell lung cancer (NSCLC) subtypes.

- **Biological Separation:** The first principal component (PC1) accounts for **62.9%** of the variance, effectively separating the NCI-H1975 samples (left) from the HCC827 samples (right). We could associate the biological differences in transcript expression among these cell lines as the primary drivers of variation in the dataset.
- **Intra-group Variability:** PC2, explaining **15.1%** of the variance, highlights intra-group heterogeneity. Notably, the replicate **R2_HCC827** is spatially distant from its counterparts (R1 and R3) on the plot, indicating potential technical variability or biological noise in that specific library. We should monitor that during the next downstream analysis.

```
pca <- plotBambu(se.multiSample, type = "pca")
pca
```

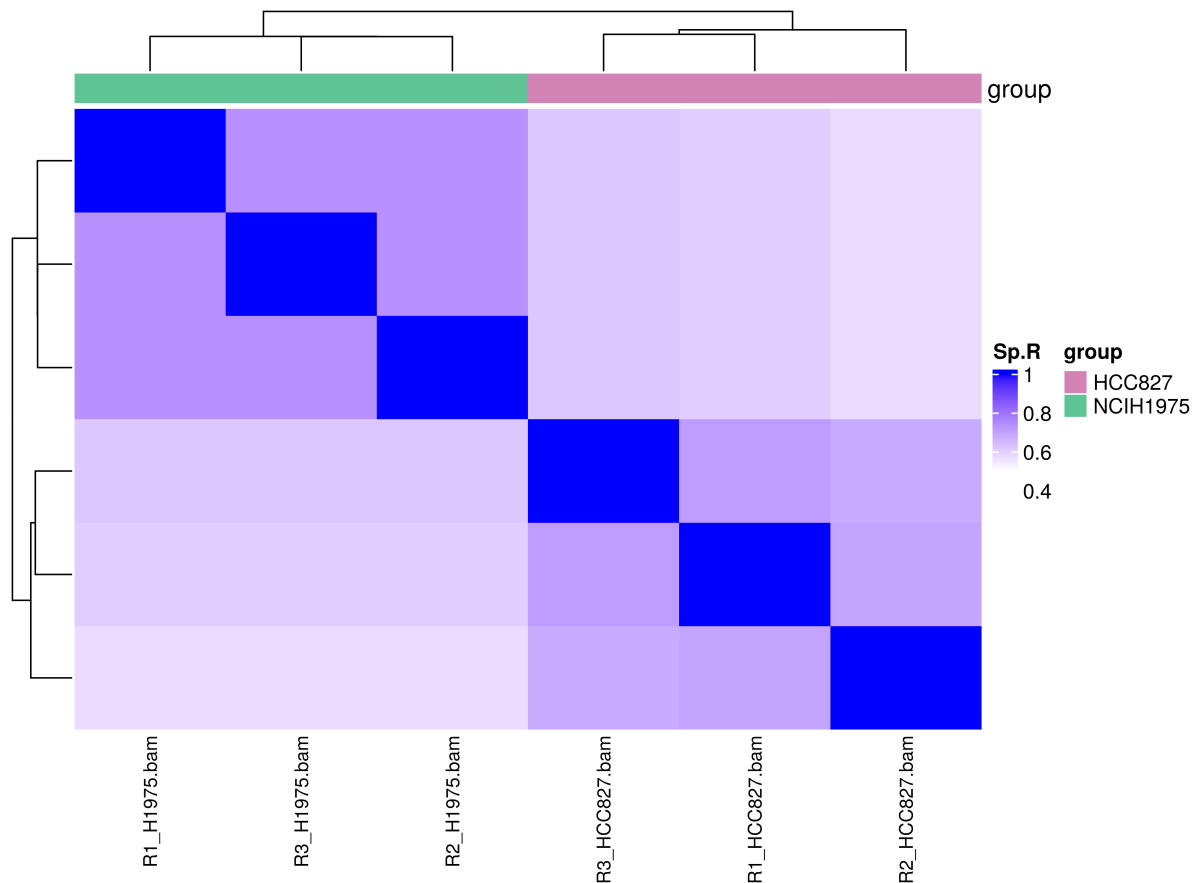


Principal Component Analysis.

Transcript-level Spearman Correlation Analysis

The Spearman's rank correlation heatmap at the transcript level shows high reproducibility between replicates (~0.8–1.0), though slightly lower than gene-level metrics. This is expected in lrrNA-seq, as allocating reads to specific isoforms is more computationally challenging than assigning them to a gene locus. The distinction between groups remains robust, with lower correlation scores (~0.4–0.6) observed between the different cell lines.

```
heatmap <- plotBambu(se.multiSample, type = "heatmap")
heatmap
```



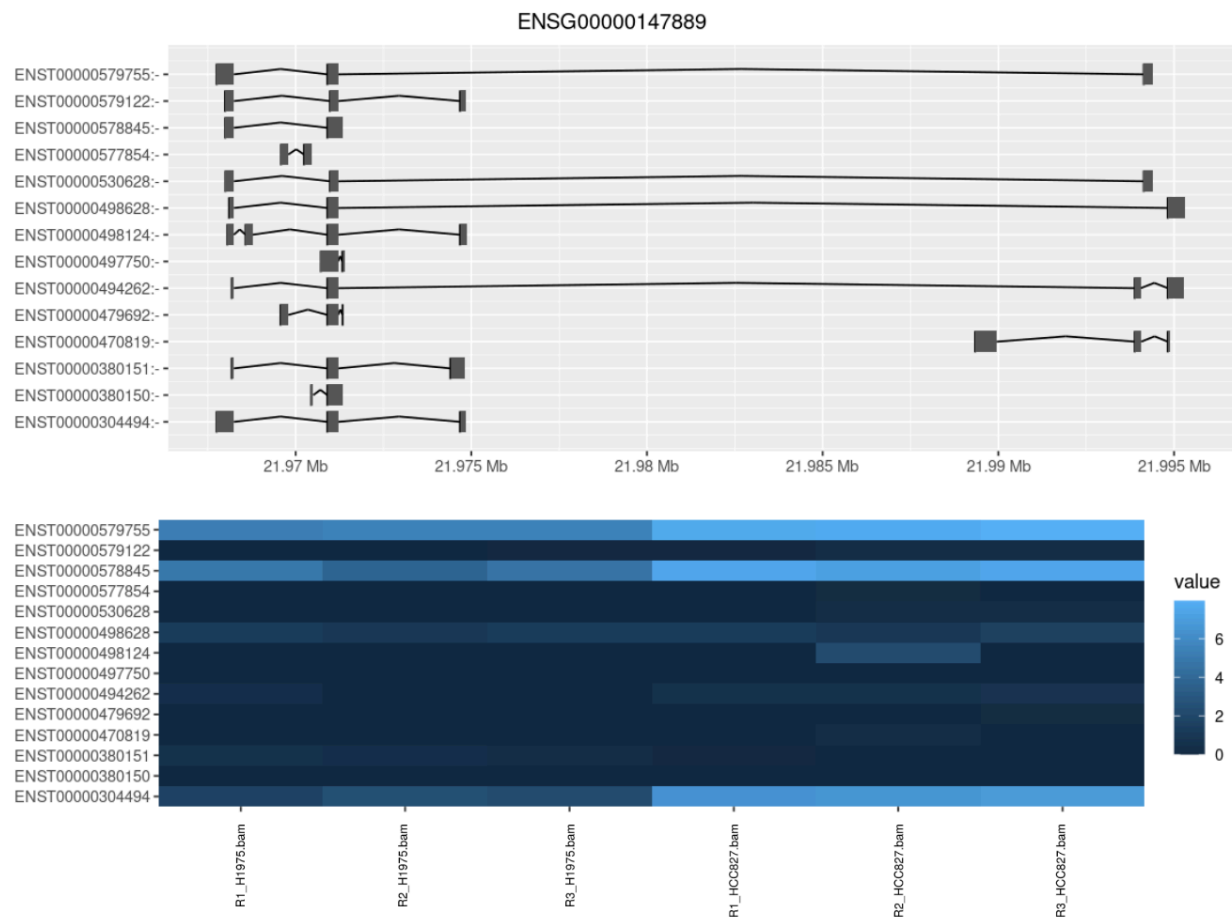
Transcript-level expression matrix based on the Spearman's rank correlation coefficient between samples.

How is the expression estimation for a specific gene, such as CDKN2A?

The annotation plot for the *CDKN2A* gene (ENSG00000147889) exemplifies Bambu's ability to resolve complex isoform structures.

- **Isoform Diversity:** Bambu identified multiple transcripts for *CDKN2A*.
- **Differential Usage:** The visualization (blue heatmap track below the gene models) suggests differential isoform usage between the groups. Certain transcripts appear more abundant in the H1975 group compared to HCC827, while others show varying expression levels even among replicates. This highlights the advantage of lrrNA-seq in capturing the full complexity of alternative splicing that might be missed by short-read quantification.

```
annotation_plot <- plotBambu(se.multiSample, type = "annotation", gene_id = "ENSG00000147889")
annotation_plo
```

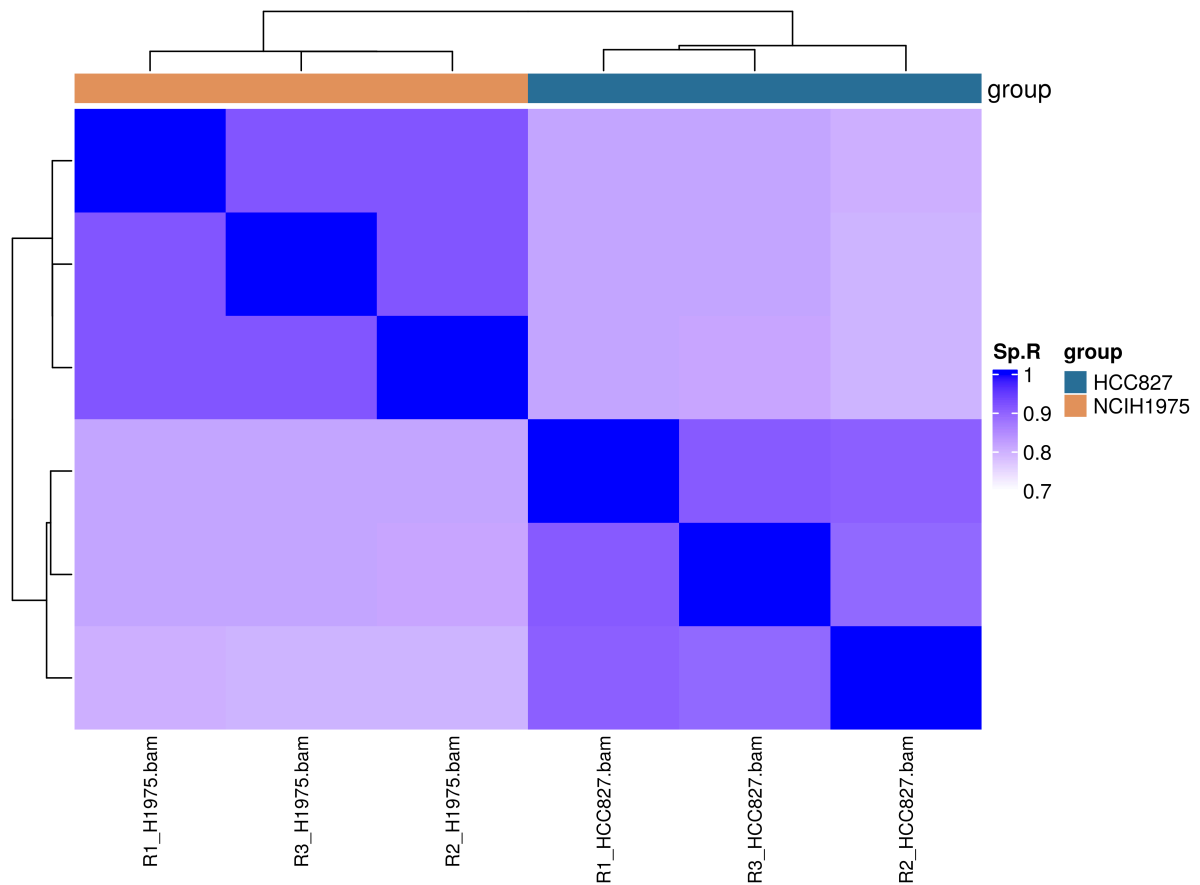


Isoform log2CPM expression estimations among samples. Different isoforms of the **CDKN2A** gene can be found expressed among the samples, including different isoforms being expressed on specific and even in the same samples.

Gene-level Spearman Correlation Analysis

When aggregated to the gene level, the correlation strengthens (mostly >0.9 within groups). This suggests that while isoform usage may fluctuate or be harder to pinpoint due to read assignment ambiguity, the overall gene expression output remains highly stable across replicates.

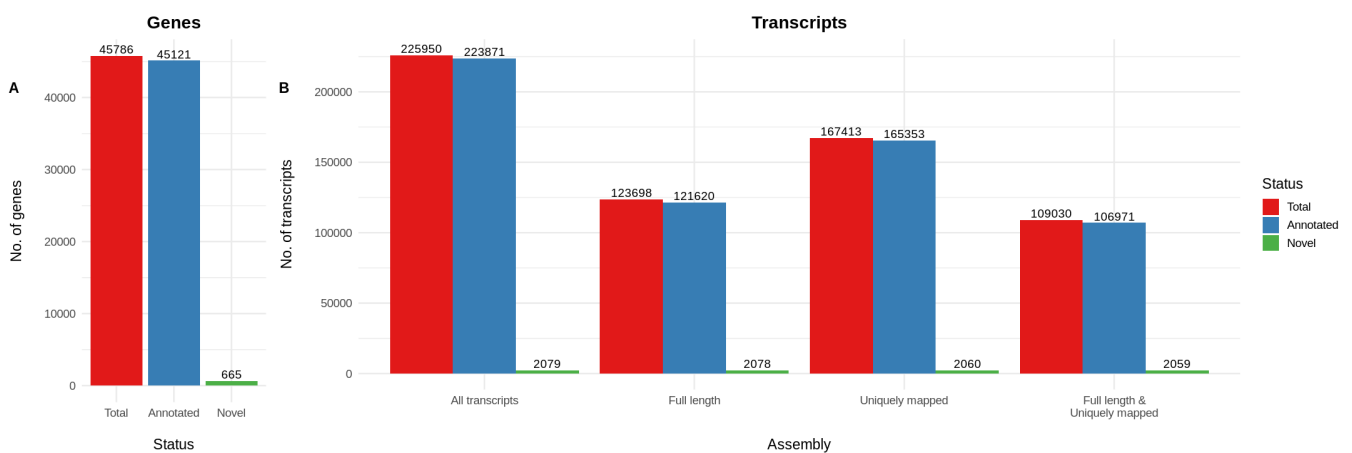
```
heatmap2 <- plotBambu(seGene.multiSample, type = "heatmap")
heatmap2
```



Gene-level expression matrix based on the Spearman's rank correlation coefficient between samples.

6. Assembly overview

We can evaluate the assembly performance by calculating total number of genes and transcripts, and then evaluating some characteristics about them which are related to the reference genome and annotation. See the script [bambu_overview.qmd](#) to understand how do we handle the Bambu Outputs in order to reach this final visualization.



Transcriptome assembly status overview

Overall results:

- Almost every single novel transcript found (99%) is supported by reads that cover the entire isoform and map uniquely to it.
- ~48% of the known transcriptome is supported by unique full-length reads. For the other ~52%, Bambu found evidence (partial reads or ambiguous reads) and used the reference annotation to “fill in the gaps.”

References

1. Dong, X. *et al.* [Benchmarking long-read RNA-sequencing analysis tools using in silico mixtures](#). *Nature Methods* **20**, 1810–1821 (2023).
2. Phelps, R. M. *et al.* [NCI-navy medical oncology branch cell line data base](#). *Journal of Cellular Biochemistry* **63**, 32–91 (1996).
3. Virmani, A. K. *et al.* [Allelotyping demonstrates common and distinct patterns of chromosomal loss in human lung cancer types](#). *Genes, Chromosomes and Cancer* **21**, 308–319 (1998).
4. Chen, Y. *et al.* [Context-aware transcript quantification from long-read RNA-seq data with Bambu](#). *Nature Methods* **20**, 1187–1195 (2023).